

Data Engineering

Introduction

Ce projet vise à récupérer et exploiter les principales informations des meilleures séries classées par Allociné. L'objectif est de stocker ces données dans une base de données Mongo, de les afficher sur un site web interactif en utilisant "Dash" et de représenter graphiquement les pays d'origine des séries à l'aide d'une carte choroplète. Le projet s'exécute au sein de containers Docker pour assurer une meilleure portabilité et gestion des dépendances.

Comment lancer le projet

1. Lancer docker

2. Créer une image mongo:

```
docker run -d --name mongoddb -p 27017:27017 mongo:latest
```

3. Lancer un container à partir de cette image:

```
docker run -d --name mongoddb -p 27017:27017 -v mongo_data:/data/db mongo:latest
```

4. Lancer l'application:

```
python main.py
```

5. Une fois l'application démarrée, retrouver l'application à cette adresse:

```
http://127.0.0.1:8050/
```

Justification des choix techniques

1. Scraping avec BeautifulSoup et Requests

Nous avons choisi **BeautifulSoup** et **Requests** pour le scraping car :

- **Simplicité**: BeautifulSoup est facile à utiliser pour extraire les données HTML.
- **Contrôle sur les requêtes** : Requests permet d'envoyer des requêtes HTTP avec des headers dynamiques pour contourner les blocages.
- **Autres techniques** : Selenium a été jugé trop lourd car il nécessite un navigateur pour exécuter les scripts.

2. Stockage des données avec MongoDB

Nous avons opté pour **MongoDB** plutôt qu'une base relationnelle car :

- **Flexibilité** : Le format JSON facilite l'insertion et la manipulation des données scannées avec un système de clé-valeur.
- **Scalabilité** : MongoDB gère bien de grandes quantités de données.
- **Autres techniques** : MySQL nécessitent un schéma rigide ce qui complique l'ajout de nouvelles informations dynamiques.

3. Développement de l'interface avec Dash

Le choix de **Dash** pour l'application web repose sur :

- **Simplicité** : Dash permet de créer des interfaces interactives sans écrire beaucoup de JavaScript. De plus, c'est une technologie qu'on a déjà utilisé pour un autre projet.
- **Visualisation** : Intégration facile avec Plotly pour la carte choroplèthe.
- **Alternatives écartées** : Flask

Méthodologie

1. Scraping des données

Le scraping a été réalisé à partir du site Allociné en utilisant **BeautifulSoup** et **Requests** pour extraire les informations suivantes :

- Titre
- Classement
- Note moyenne des utilisateurs/Presse
- Genres
- Créateurs
- Acteurs principaux
- Pays d'origine
- URL

Code:

Nous avons développé une fonction `“get_series_info(url)”` qui récupère les informations des séries listées sur Allociné en utilisant **requests** pour l'extraction du contenu HTML et **BeautifulSoup** pour le parsing. Les détails de la série sont ensuite extraits et stockés dans une structure de données. Une autre fonction `“get_series_details(url)”` permet de récupérer les détails d'une série, notamment son pays d'origine.

Le script assure une gestion des erreurs et inclut des en-têtes HTTP dynamiques pour éviter d'être bloqué par les protections anti-scraping.

2. Stockage des données

Nous avons utilisé **MongoDB** pour stocker les informations extraites. Nous avons choisi cela pour faciliter l'insertion et la gestion des données à l'image d'un dictionnaire sur python. Voici un aperçu des collections utilisées :

- **series** (titre, classement, genres, créateurs, acteurs, note presse, note spectateurs, lien, pays)

Insertion dans la base de données

Nous avons mis en place une fonction "*insert_series_db()*" qui récupère les données scrappées et les insère dans MongoDB. Cette fonction supprime d'abord les anciennes données avant d'insérer de nouvelles entrées, garantissant ainsi une mise à jour propre des informations et pas de doublons.

Une autre fonction "*get_series_from_db()*" permet de récupérer les séries stockées pour les afficher sur le site web.

3. Affichage des données

Nous avons choisi **Dash** pour le développement de l'interface web interactive. L'interface utilisateur comprend :

- Un tableau interactif affichant les séries avec la possibilité de **filtrer et trier** les résultats.
- Une **barre de recherche** permettant de trouver une série par titre, acteur ou créateur.
- Une **carte choroplèthe** affichant la distribution des pays d'origine des séries.

Implémentation du routeur de pages

Le routeur est géré avec "*dcc.Location*" et permet de naviguer entre les différentes sections du site. Nous avons défini les pages suivantes :

- *create_home_page()* pour l'accueil
- *create_series_page()* pour l'affichage des séries
- *create_map_page()* pour la carte interactive

Un callback met à jour dynamiquement le contenu affiché en fonction de l'URL actuelle et ajuste le style des liens pour indiquer la page active.

Affichage du tableau des séries

Nous avons créé la fonction "*format_data(series_data)*", qui transforme les données stockées en un format lisible pour l'affichage dans un tableau interactif. Cette fonction assure également la correction des liens vers Allociné et la gestion des valeurs manquantes.

Difficultés rencontrées

- **Données incomplètes** : certains champs ont nécessité un prétraitement spécifique.

- **Conteneur Docker:** Nous n'avons pas réussi à conteneuriser notre application dash, car elle ne se connecte pas à notre conteneur MongoDB

Conclusion

Ce projet de Data Engineering nous a permis d'explorer plusieurs aspects clés de l'acquisition, du stockage et de la visualisation de données en environnement conteneurisé. En combinant **le scraping web avec BeautifulSoup et Requests**, **le stockage dans MongoDB**, et **l'affichage interactif avec Dash**, nous avons mis en place une solution pour explorer et analyser les meilleures séries d'Allociné. Ce projet nous a donc permis de renforcer nos compétences en **scraping, gestion de bases de données, développement d'interfaces interactives et conteneurisation**.